



UNIVERSIDAD DE ALMERÍA

PROCESOS DE INGENIERÍA DEL SOFTWARE 2

Documentación, LPI y RGPD

GRUPO 7

Andrés Ruiz Andújar

Sergio Gómez Vico

Adrián Martínez Granados

Raúl Martínez Gutiérrez

Índice de Contenidos

1. Introducción.....	4
2. Ingeniería de Documentación.....	4
2.1. Filosofía Docs as Code.....	4
2.1.1. ¿Qué es Docs as Code?.....	4
2.1.2. El problema de Word y otros editores de texto convencionales.....	4
2.1.3. Ventajas y desventajas de la filosofía	5
2.1.4. Como implementar Docs as Code	6
2.2. Estándar ISO/IEC/IEEE 26514	7
2.2.1. Definición y Contexto de la ISO 26514	7
2.2.2. Bases para la documentación de calidad	7
2.2.3. Integración en el ciclo de desarrollo.....	7
2.2.4. Análisis de audiencia	8
2.3. Mantenibilidad e ISO 25010.....	8
2.3.1. La mantenibilidad en ISO 25010.....	8
2.3.2. El “Factor bus”	9
2.3.3. Estrategias y buenas prácticas	10
2.4. Documentación de APIs	11
2.4.1. Qué es OpenAPI y Swagger	11
2.4.2. Contrato técnico.....	11
2.4.3. El contrato legal e infranqueable entre partes	12
2.4.4. Beneficios prácticos.....	13
2.4.5. Relación con ISO 25010.....	14
3. Blindaje Legal y Ético	14
3.1. Propiedad Intelectual (LPI)	15
3.1.1. ¿Qué es la LPI?.....	15
3.1.2. ¿Qué protege la LPI?	15
3.1.3. Tipos de derechos de autor	15
3.1.4. Copyright vs Patente.....	16
3.1.5. Infracciones y consecuencias	17
3.2. RGPD	17
3.2.1. ¿Qué es el RGPD?	17
3.2.2. Principios fundamentales.....	17

3.2.3. Derechos ARSULIPO	18
3.2.4. ¿Cómo cumplir el RGPD?	19
3.2.5. Ventajas de cumplir el RGPD	19
3.3. Cumplimiento de Licencias (ISO/IEC 5230)	19
3.3.1. Tipos básicos de licencias de software	20
3.3.2. Licencias Permisivas y su impacto comercial	20
3.3.3. Licencias Copyleft y la protección del código derivado	21
3.3.4. Derechos de autor y marcos legales de protección.....	21
3.3.5. Gestión de riesgos y toma de decisiones	22
3.4. Ética y Responsabilidad Civil: ISO 42001	22
3.4.1. Fundamentos del SGIA: Gestión y Gobernanza	23
3.4.2. Desafíos Técnicos de la IA	23
3.4.3. Impacto y aplicabilidad	23
3.4.4. Responsabilidad Civil del Ingeniero y Trazabilidad Legal (ByDesign).....	24
4. La documentación como línea de defensa	24
4.1. Docs as Code e ISO 26514 como base del cumplimiento	24
4.2. Las licencias, la LPI y el RGPD no se gestionan de memoria	25
4.3. El factor humano: el bus factor y la responsabilidad civil	25
5. Conclusión.....	26
6. Bibliografía.....	27
7. Anexo	28
7.1. Documento de Arquitectura ISO 26514	28
7.2. Contrato de Cesión de Derecho	30

1. Introducción

El desarrollo de software moderno no se limita a escribir código que funcione. Implica documentar lo que se construye, cumplir con marcos legales que regulan su uso y distribución, y hacerlo de forma ética y responsable. Este documento aborda esas tres dimensiones de forma estructurada, con el objetivo de ofrecer una visión práctica y aplicable para equipos de ingeniería del software.

La primera parte trata la ingeniería de documentación: cómo se escribe, organiza y mantiene la documentación técnica de forma profesional, desde la filosofía Docs as Code hasta los estándares internacionales que la regulan. La segunda parte aborda el blindaje legal y ético: qué marcos normativos afectan al software que se desarrolla y distribuye, y cómo cumplirlos sin que eso se convierta en una carga burocrática.

2. Ingeniería de Documentación

La documentación es una parte del producto, no un añadido opcional que se completa al final del proyecto. Sin embargo, en la práctica, suele tratarse como una tarea secundaria que se delega en quien tenga tiempo libre o se redacta en el último sprint antes de la entrega. El resultado es documentación desactualizada, incompleta y de escaso valor para quien la necesita.

Esta sección aborda la documentación desde una perspectiva de ingeniería: con procesos, estándares y herramientas que permiten producirla con la misma disciplina con la que se escribe el código. Se exploran tres ejes complementarios: la filosofía Docs as Code, el estándar ISO 26514, la mantenibilidad según ISO 25010 y la documentación de APIs como caso de uso concreto.

2.1. Filosofía Docs as Code

2.1.1. ¿Qué es Docs as Code?

Es una filosofía que sostiene que se debe escribir y tratar la documentación con las mismas herramientas y procesos que el código fuente. Ello implica el seguimiento de los mismos flujos de trabajo que los equipos de desarrollo. Cuando se pone en práctica, los medios con los que se crea la documentación viran desde los documentos de Word o blocs de notas tradicionales, que se sostienen en base a revisiones sistemáticas manuales, con procesos poco satisfactorios; hacia el cercano texto plano que representa Markdown, cuyo cuidado se cimenta sobre Git, usando pull request, y todo lo que tiene que ver con la integración automática.

2.1.2. El problema de Word y otros editores de texto convencionales

Los procesadores de texto tradicionales como Word fueron diseñados para documentos estáticos orientados a la impresión, no para entornos de desarrollo colaborativo y en continua evolución. Esto genera varios problemas estructurales cuando se usan para documentar software:

- **Control de versiones manual e ineficiente.** Sin integración con sistemas como Git, los equipos recurren a convenciones de nombre de archivo (doc_v2_final_DEFINITIVO.docx) que son frágiles, propensas a errores y difíciles de auditar.
- **Colaboración bloqueante.** Dos personas no pueden editar el mismo documento simultáneamente sin riesgo de sobreescritura o conflictos, lo que ralentiza los flujos de trabajo en equipo.
- **Formatos binarios opacos.** Los archivos .docx no son legibles como texto plano, lo que impide hacer revisiones de cambios línea a línea, comparaciones (diffs) o integraciones automáticas en pipelines de CI/CD.
- **Desincronización con el código.** La documentación vive en un repositorio o carpeta separada del código fuente, lo que favorece que quede desactualizada con cada nueva versión del software.
- **Dependencia de herramientas propietarias.** Abrir, editar y renderizar el documento correctamente depende de tener instalado el software adecuado, introduciendo fricciones innecesarias en equipos heterogéneos.

2.1.3. Ventajas y desventajas de la filosofía

Como toda filosofía de trabajo, Docs as Code no es una solución universal. Aporta mejoras claras sobre los flujos tradicionales, pero también exige un esfuerzo de adaptación que conviene tener en cuenta antes de adoptarla.

Entre sus ventajas, encontramos:

- **Trazabilidad completa.** Al usar Git, cada cambio en la documentación queda registrado con autor, fecha y motivo, igual que cualquier commit de código.
- **Revisión estructurada.** Las pull requests permiten revisar y aprobar cambios en la documentación antes de publicarlos, aplicando el mismo estándar de calidad que al código.
- **Automatización.** La documentación puede generarse, validarse y desplegarse automáticamente mediante pipelines de CI/CD, eliminando pasos manuales y reduciendo el error humano.
- **Colaboración sin bloqueos.** Al trabajar sobre texto plano en ramas separadas, varios autores pueden contribuir en paralelo y resolver conflictos con las mismas herramientas que ya conocen.
- **Portabilidad y durabilidad.** El texto plano en Markdown no depende de ninguna aplicación propietaria y puede convertirse a HTML, PDF u otros formatos según necesidad.

Por otro lado, y para tener un juicio justo de cara a valorar esta filosofía, es preciso tomar en cuenta las siguientes desventajas:

- **Curva de aprendizaje inicial.** Los perfiles no técnicos (redactores, responsables de producto, equipos legales) pueden encontrar Git y Markdown poco intuitivos frente a un editor visual como Word.
- **Limitaciones de formato complejo.** Markdown no cubre de forma nativa layouts complejos, tablas muy elaboradas o elementos visuales ricos que sí admite un procesador de texto convencional.
- **Configuración inicial costosa.** Establecer el pipeline de publicación, las herramientas de generación (MkDocs, Hugo, Sphinx) y las convenciones del equipo requiere una inversión de tiempo significativa al principio.
- **Resistencia organizativa.** En organizaciones con procesos establecidos alrededor de Word o Confluence, la transición a Docs as Code implica un cambio cultural que puede generar fricción.

2.1.4. Como implementar Docs as Code

La implementación no tiene por qué hacerse de golpe. Lo más razonable es un enfoque incremental que no obligue a reescribir todo desde el primer día ni a paralizar el trabajo del equipo.

1. **Adoptar Markdown como formato estándar.** Empezar a escribir la documentación nueva en archivos .md. La documentación existente puede migrarse progresivamente.
2. **Incorporar la documentación al repositorio.** Almacenar los archivos junto al código en el mismo repositorio Git, en una carpeta /docs, de modo que la documentación viaje con cada versión del software.
3. **Establecer un flujo de revisión mediante pull requests.** Cualquier cambio debe pasar por una rama y al menos una revisión antes de integrarse, igual que el código.
4. **Configurar una herramienta de generación estática.** Herramientas como MkDocs, Hugo o Sphinx transforman los archivos Markdown en un sitio web navegable que puede publicarse automáticamente.
5. **Integrar la documentación en el pipeline de CI/CD.** Añadir pasos que validen la sintaxis, detecten enlaces rotos y desplieguen la documentación actualizada en cada merge a la rama principal.
6. **Definir convenciones del equipo.** Establecer guías de estilo y criterios de revisión para que la documentación sea coherente independientemente de quién la escriba.

2.2. Estándar ISO/IEC/IEEE 26514

Plantilla de documento de Arquitectura según ISO/IEC/IEEE 26514:2022 en Anexo.

2.2.1. Definición y Contexto de la ISO 26514

La ISO/IEC/IEEE 26514:2022 es el estándar básico revisado para el diseño y la producción de información de usuario para software y sistemas. Su edición de 2022 actualiza y reemplaza la versión de 2008, y se basa en la DIN EN IEC/IEEE 82079-1, adoptando sus tres pilares fundamentales. La diferencia entre ambas es de alcance: la 82079-1 es más general y aplica a cualquier tipo de producto, mientras que la 26514 se especializa específicamente en software y sistemas.

Su filosofía se basa en que todo el proceso de documentación debe estar orientado estrictamente a las necesidades del público objetivo. Esta idea ya estaba presente desde 2006 en la práctica editorial, pero la norma la formaliza y sistematiza. Es importante entender que no es una norma de "cumplir por cumplir": su valor está en poner al usuario final en el centro de todo el proceso documental, no en el cumplimiento formal de sus requisitos.

2.2.2. Bases para la documentación de calidad

La ISO 26514 establece tres principios básicos para considerar que la información de usuario es de calidad. Los principios son:

1. **Análisis.** Estudiar en profundidad los públicos objetivo y las actividades concretas que realizarán con el producto. Sin este paso, el resto no tiene base.
2. **Derivación.** Crear conceptos de información que nazcan directamente del análisis anterior. El contenido no se inventa: se deriva de lo que el usuario necesita hacer.
3. **Validación.** Asegurar la calidad mediante comprobaciones de usabilidad para confirmar o corregir las suposiciones de diseño tomadas en las etapas previas.

Estos tres principios son el argumento principal contra el manual en Word de última hora: si no hay análisis previo del usuario, no hay documentación de calidad según la norma.

2.2.3. Integración en el ciclo de desarrollo

La norma defiende que la documentación técnica debe crearse al mismo tiempo que el software, no al finalizar el desarrollo. La documentación tardía es documentación desactualizada desde el inicio.

Para lograrlo, subraya la necesidad de una interconexión interdisciplinar temprana entre desarrolladores, escritores técnicos, diseñadores UX y responsables de producto. Esto conecta directamente con la filosofía Docs as Code vista en el punto anterior: la documentación integrada en el pipeline de CI/CD es precisamente la implementación técnica de este principio.

2.2.4. Análisis de audiencia

A diferencia de otras normas que usan categorías vagas como "personal técnico" o "usuario final", la ISO 26514 promueve el uso de perfiles de destinatarios concretos y reales, eliminando ambigüedades en el diseño de la documentación.

Cuando el público objetivo es muy variado, la norma recomienda la estrategia de divulgación progresiva: el contenido básico está siempre visible y accesible para todos, mientras que el detalle técnico solo se accede mediante enlaces, menús desplegables o secciones expandibles. Esto evita sobrecargar a los usuarios simples y frustrar a los avanzados. Un ejemplo práctico es la documentación de APIs con Swagger (Véase 2.4. Documentación de APIs), donde los endpoints básicos están al frente y los parámetros avanzados están colapsados.

2.3. Mantenibilidad e ISO 25010

La mantenibilidad es uno de los pilares fundamentales de la calidad del software. Un sistema que no puede ser comprendido, modificado o probado de forma eficiente acumula deuda técnica, eleva costes de operación y pone en riesgo la continuidad del negocio. La norma ISO/IEC 25010 proporciona el marco de referencia internacional para medir y garantizar esta característica de forma objetiva y auditable.

2.3.1. La mantenibilidad en ISO 25010

La **ISO/IEC 25010:2011** (actualizada en 2023) define el modelo de calidad del producto software. Dentro de sus ocho características principales, la **Mantenibilidad** describe la capacidad de un sistema para ser modificado de manera efectiva y eficiente sin introducir defectos ni degradar la calidad existente.

Las cinco subcaracterísticas de la Mantenibilidad son:

Subcarasterística	Definición	Indicadores Clave
Modularidad	El sistema se divide en componentes discretos de forma que un cambio en uno tenga el mínimo impacto en los demás.	Alta cohesión, bajo acoplamiento. Uso de arquitectura por capas o microservicios.
Reusabilidad	Los activos pueden ser utilizados por más de un sistema o para construir otros activos.	Librerías internas, componentes genéricos, patrones de diseño.
Anazabilidad	Facilidad para diagnosticar deficiencias, causas de fallos o las partes a modificar.	Logs estructurados, trazabilidad, herramientas de profiling.
Modificabilidad	El producto puede ser modificado de forma efectiva sin introducir defectos.	Código limpio, refactorización continua, pruebas de regresión.
Testeabilidad	El software puede ser sometido a pruebas para validar cambios de forma eficiente.	Cobertura de tests, inyección de dependencias, mocks y stubs.

La mantenibilidad no es un atributo binario (mantenible / no mantenible); se mide con métricas objetivas como el índice de mantenibilidad de Halstead, la complejidad ciclomática de McCabe o el tiempo medio de reparación (MTTR). La documentación técnica incide directamente sobre la Analizabilidad y la Modificabilidad.

2.3.2. El “Factor bus”

El nombre viene de una pregunta incómoda: **¿cuántas personas de tu equipo tendrían que ser atropelladas por un autobús para que el proyecto colapsara?**

Si la respuesta es "una", tienes un Factor Bus de 1. Eso significa que hay una sola persona que sabe cómo funciona una parte crítica del sistema, y si esa persona desaparece (por accidente, por enfermedad, por irse a otra empresa), ese conocimiento desaparece con ella.

También se llama **Factor Camión** o **Bus Factor** en inglés, y a veces se suaviza como **Factor Lotería** (¿qué pasa si alguien se gana la lotería y deja el trabajo mañana?).

Factor Bus	Nivel de Riesgo	Consecuencia	Acción Recomendada
1	Crítico	El proyecto colapsa si esa persona no está disponible.	Documentación urgente, pair programming, rotación de tareas.
2-3	Alto	Riesgo significativo si coincide la ausencia de dos personas clave.	Wikis técnicas, onboarding estructurado, revisiones de código.
4-6	Medio	El equipo puede absorber bajas puntuales con esfuerzo.	Mantener ADRs actualizados, pruebas documentadas.
>6	Bajo	Conocimiento distribuido. El proyecto es resiliente.	Automatizar documentación con Docs as Code.

2.3.2.1. La documentación como mitigador del Factor Bus

La documentación técnica actúa como el **sistema inmunológico** del conocimiento organizacional.

Cuando se produce correctamente y se integra en el flujo de desarrollo (filosofía *Docs as Code*), convierte el conocimiento tácito —almacenado solo en la mente de un experto— en conocimiento explícito y transferible, reduciendo el Factor Bus de forma directa y medible.

El Factor Bus ataca principalmente la **analizabilidad** y la **modificabilidad**. Un sistema con Factor Bus 1 puede tener código impecable técnicamente, pero si solo lo entiende una persona, su mantenibilidad real es pésima según la ISO 25010.

Mecanismos de reducción del Factor Bus mediante documentación:

- **Architecture Decision Records (ADR):** Documentos breves que registran el contexto, las opciones evaluadas y la decisión tomada para cada decisión arquitectónica

relevante. Permiten a cualquier nuevo miembro del equipo entender el «*por qué*» detrás del código, no solo el «*cómo*».

- **Runbooks operacionales:** Procedimientos paso a paso para tareas de operación críticas (despliegues, recuperación ante fallos, escalado). Reducen la dependencia de operadores expertos.
- **Diagramas C4 actualizados:** Representaciones visuales del sistema en cuatro niveles (Contexto, Contenedores, Componentes y Código) que permiten comprender la arquitectura de forma gradual.
- **Docstrings y comentarios de alto valor:** No explican *qué* hace el código (eso ya lo dice el código), sino *por qué* se tomó una decisión concreta, especialmente en algoritmos complejos o workarounds.
- **Changelogs semánticos:** Registro estructurado de cambios por versión (siguiendo Semantic Versioning) que facilita la trazabilidad y la auditoría.

2.3.3. Estrategias y buenas prácticas

Elevar el nivel de mantenibilidad de un proyecto requiere la combinación de prácticas técnicas, culturales y organizativas. Las siguientes estrategias están alineadas con los requisitos de la ISO/IEC 25010 y con el estándar ISO/IEC/IEEE 26514 para la documentación técnica.

Arquitectura modular y principio de responsabilidad única

Cada módulo o clase debe tener una razón única de cambio (SRP). Proyectos estructurados en capas o dominios (Domain-Driven Design) permiten modificar una parte del sistema sin efecto cascada. La documentación de la arquitectura (C4, UML) hace este diseño auditable.

Docs as Code integrado en CI/CD

La documentación debe vivir en el mismo repositorio que el código y estar sujeta a las mismas revisiones (pull requests, linting). Herramientas como MkDocs, Sphinx o Docusaurus generan sitios estáticos de documentación automáticamente en el pipeline de CI/CD, asegurando que la documentación publicada corresponde siempre al código en producción.

Revisiones de código con criterios de mantenibilidad

Los criterios de aceptación de un pull request deben incluir: cobertura de tests, ausencia de complejidad ciclomática excesiva (>10), actualización de la documentación afectada y verificación de que los ADRs correspondientes existen o han sido creados.

Métricas objetivas y dashboards de calidad

Integrar herramientas como SonarQube, CodeClimate o Radon en el pipeline para medir

automáticamente el índice de mantenibilidad, la deuda técnica acumulada y la cobertura de documentación. Estas métricas son los KPIs directos de la subcaracterística de mantenibilidad de la ISO 25010.

Cultura de conocimiento compartido

Implementar pair programming rotativo, sesiones de mob programming y talleres internos (tech talks) para distribuir el conocimiento crítico. La documentación escrita y el conocimiento compartido en persona son complementarios, no sustitutos.

Versionado semántico y changelogs estructurados

Adoptar Semantic Versioning (MAJOR.MINOR.PATCH) y el estándar Keep a Changelog garantiza que cualquier desarrollador pueda entender el historial de cambios del sistema sin necesidad de leer todos los commits de Git.

2.4. Documentación de APIs

Las **Application Programming Interfaces (APIs)** son el tejido conectivo del software moderno. Cada API es, al mismo tiempo, un *contrato técnico* entre sistemas y un *contrato legal* entre las partes que acuerdan su uso. La ausencia de documentación formal de una API no solo es un déficit técnico: es un vacío legal con implicaciones directas en la Propiedad Intelectual, la responsabilidad civil del ingeniero y el cumplimiento del RGPD.

2.4.1. Qué es OpenAPI y Swagger

La **Especificación OpenAPI (OAS)**, anteriormente conocida como Swagger, es un **estándar** abierto para describir APIs REST mediante un formato legible tanto por humanos como por máquinas (YAML o JSON). La especificación es mantenida por la *OpenAPI Initiative* bajo el paraguas de la Linux Foundation, con versiones clave 2.0 (Swagger), 3.0.x y 3.1.x.

Un documento OpenAPI completo incluye: metadata de la API (título, versión, licencia), servidores de destino, rutas (*paths*), operaciones HTTP, parámetros, esquemas de datos (reutilizables mediante \$ref), códigos de respuesta, mecanismos de autenticación (*securitySchemes*) y ejemplos. Esta completitud convierte el documento OAS en la fuente de verdad única (*Single Source of Truth*) para todos los actores del sistema.

2.4.2. Contrato técnico

En ingeniería de software, un **contrato técnico de API** es el acuerdo formal sobre el comportamiento esperado de una interfaz: qué operaciones expone, qué datos acepta, qué datos devuelve y bajo qué condiciones de error. El documento OpenAPI es la materialización escrita de este contrato.

Elementos que definen el contrato técnico en OpenAPI:

Endpoints y métodos HTTP: Cada ruta define exactamente qué operaciones (GET, POST, PUT, PATCH, DELETE) son válidas, evitando ambigüedades de comportamiento.

Schemas de entrada y salida: Los objetos JSON/XML están tipados y validados mediante JSON Schema, lo que permite validación automática de contratos (*contract testing*) con herramientas como Pact o Dredd.

Códigos de respuesta: Cada operación documenta los códigos HTTP posibles (200, 201, 400, 401, 404, 422, 500) y el significado semántico de cada uno, evitando interpretaciones divergentes entre consumidor y proveedor.

Versionado de API: La especificación incluye la versión de la API (v1, v2) y permite mantener múltiples versiones simultáneas, garantizando retrocompatibilidad contractual.

SLA implícito: Los parámetros de paginación, límites de petición (rate limits) y timeouts documentados en el OAS definen expectativas de rendimiento que pueden convertirse en requisitos no funcionales auditables.

2.4.3. El contrato legal e infranqueable entre partes

Más allá de su dimensión técnica, la documentación OpenAPI de una API tiene un valor jurídico directo cuando se incorpora a acuerdos contractuales entre empresas. En el contexto de la **Ley de Propiedad Intelectual (LPI)** española y del marco europeo, el documento OAS puede funcionar como:

Evidencia de la obra original: El documento OAS, al estar versionado en Git con timestamps verificables, constituye prueba fehaciente de la autoría y fecha de creación de la interfaz, relevante ante reclamaciones de copia no autorizada (Art. 96 LPI sobre programas de ordenador).

Anexo técnico de contratos de desarrollo: En contratos de desarrollo a medida, el OAS actuado como anexo técnico vinculante define exactamente qué entrega el proveedor y qué espera el cliente. Su incumplimiento genera responsabilidad contractual directa.

Cesión de derechos de uso: El documento OAS puede incluir una sección de licencia (campo info.license) que especifica los términos bajo los cuales terceros pueden consumir la API, actuando como un contrato de adhesión digital.

Trazabilidad para el RGPD: Los schemas del OAS que incluyen campos de datos personales (nombre, email, IP) deben estar alineados con el Registro de Actividades de Tratamiento (RAT), conectando la arquitectura técnica con las obligaciones legales del Reglamento.

Evidencia en litigios por incumplimiento de SLA: Si el comportamiento documentado en el OAS difiere del comportamiento real de la API, el documento sirve como prueba del estándar de servicio comprometido.

2.4.4. Beneficios prácticos

La adopción de OpenAPI como estándar de documentación de APIs genera beneficios tangibles y medibles en cuatro dimensiones: velocidad de desarrollo, calidad, cumplimiento normativo y gestión del riesgo.

Generación automática de código

El documento OAS permite generar automáticamente SDKs de cliente en más de 40 lenguajes y stubs de servidor, reduciendo el tiempo de integración de semanas a horas y eliminando errores de interpretación manual.

Testing automatizado de contratos

Herramientas como Dredd o Schemathesis generan casos de prueba automáticamente a partir del documento OAS, verificando que la implementación cumple con el contrato publicado en cada ejecución del pipeline CI/CD.

Portal de desarrollador auto-generado

Swagger UI y herramientas como Redocly o Stoplight generan portales de documentación interactivos donde los consumidores de la API pueden explorar y probar los endpoints sin necesidad de documentación adicional.

Mock servers para desarrollo paralelo

A partir del OAS, herramientas como Prism o WireMock generan servidores mock que devuelven respuestas simuladas, permitiendo que el equipo de frontend desarrolle en paralelo al backend desde el primer día.

Validación de seguridad

Herramientas de seguridad como OWASP ZAP o 42Crunch pueden escanear automáticamente el documento OAS en busca de vulnerabilidades de seguridad en el diseño de la API antes de que sea implementada.

Gobierno de APIs a escala

En organizaciones con múltiples equipos, el OAS centralizado en un registro de APIs permite aplicar políticas de estilo, detectar duplicidades y garantizar la coherencia de contratos a nivel de empresa.

Actividad	Sin OpenAPI	Con OpenAPI	Reducción de Esfuerzo
Generación de SDK cliente	3–5 días de desarrollo manual	2–4 horas con Codegen	90%
Onboarding de consumidores de API	1–2 semanas con docs manuales	1–2 días con Swagger UI	80%
Detección de breaking changes	En producción (coste alto)	En CI/CD (coste cero)	100%
Auditoría de seguridad de diseño	Manual, tardía y costosa	Automatizada en pipeline	70%

Creación de tests de integración	Manual por desarrollador	Auto-generados desde OAS	60%
----------------------------------	--------------------------	--------------------------	-----

2.4.5. Relación con ISO 25010

La documentación OpenAPI no solo aporta valor práctico; impacta directamente sobre múltiples características del modelo de calidad definido por la **ISO/IEC 25010**, no solo sobre la Mantenibilidad. Esta conexión hace que el OAS sea un instrumento de calidad transversal y auditable.

Característica ISO 25010	Impacto del documento OpenAPI	Relevancia
Analizabilidad (Mantenibilidad)	El OAS describe la estructura completa de la API, facilitando el diagnóstico de fallos y el impacto de cambios sin necesidad de leer el código fuente.	Alta
Modificabilidad (Mantenibilidad)	Las versiones del OAS en Git permiten comparar contratos entre versiones, identificando breaking changes antes de implementar.	Alta
Adaptabilidad (Potabilidad)	Un OAS estandarizado permite migrar la implementación de la API (cambio de lenguaje, framework o proveedor cloud) sin cambiar el contrato.	Alta
Interoperabilidad (Compatibilidad)	El OAS actúa como lengua franca entre sistemas heterogéneos, garantizando que cualquier cliente que cumpla el contrato puede integrarse independientemente de su stack tecnológico.	Muy alta
Integridad (Seguridad)	Las secciones securitySchemes del OAS documentan los mecanismos de autenticación (OAuth2, API Keys, JWT), haciendo auditable la política de seguridad de la API.	Alta
Tolerancia a Fallos (Fiabilidad)	Los códigos de error documentados en el OAS permiten diseñar estrategias de manejo de errores y circuit breakers en los clientes.	Media
Aprendizaje (Usabilidad)	Swagger UI reduce drásticamente la curva de aprendizaje para nuevos consumidores de la API, convirtiendo la documentación en una experiencia interactiva.	Alta

La **ISO/IEC/IEEE 26514:2022** exige que la documentación de usuario y de sistema sea completa, precisa, coherente y mantenible. El documento OpenAPI satisface estos cuatro requisitos para la dimensión de la API: es generado automáticamente desde el código (precisión), versionado en Git (coherencia y mantenibilidad) y cubre todos los endpoints del sistema (completitud). Su uso debería ser obligatorio en cualquier proyecto auditado bajo esta norma.

3. Blindaje Legal y Ético

Desarrollar software implica moverse dentro de un marco legal que regula los derechos sobre el código, el tratamiento de los datos que maneja y las condiciones bajo las que puede distribuirse. Ignorar ese marco no exime de responsabilidad: las consecuencias van desde sanciones económicas hasta responsabilidad civil directa sobre los ingenieros que firman el software.

Esta sección aborda los cuatro pilares del blindaje legal y ético en el desarrollo de software: la propiedad intelectual, la protección de datos, el cumplimiento de licencias y la ética en el uso de inteligencia artificial. El objetivo no es ofrecer asesoramiento jurídico, sino proporcionar el conocimiento mínimo necesario para tomar decisiones informadas y documentadas.

3.1. Propiedad Intelectual (LPI)

Véase plantilla de Contrato para Cesión de Derechos de Software en el Anexo

3.1.1. ¿Qué es la LPI?

La LPI es la Ley de Propiedad Intelectual en España. Es la norma que protege los derechos de los autores sobre sus creaciones. Su función principal es regular quién puede usar una obra, en qué condiciones y qué derechos tiene la persona que la ha creado. Por ejemplo, el autor puede decidir si su obra se publica, si se copia, si se modifica, si se distribuye o si otra persona puede utilizarla.

El objetivo principal de la LPI es proteger la creación intelectual y evitar que otras personas usen obras ajenas sin permiso. De esta forma, se reconoce el esfuerzo del autor y se garantiza que pueda controlar el uso de su trabajo.

3.1.2. ¿Qué protege la LPI?

La LPI protege las obras originales creadas por una persona, siempre que esa creación esté expresada de una forma concreta. Esto significa que la ley protege el resultado final de una creación, no simplemente una idea que alguien tiene en la cabeza.

Entre las obras protegidas se incluyen las obras literarias, artísticas, musicales, audiovisuales, científicas y los programas de ordenador. En el ámbito de la informática, esto es importante porque el código fuente, la documentación técnica o ciertos contenidos digitales pueden estar protegidos por derechos de autor.

3.1.3. Tipos de derechos de autor

El autor es la persona física que crea la obra. Por ejemplo, quien escribe un libro, compone una canción o hace una fotografía.

- **Derechos morales (Personales):** Los derechos morales son los derechos personales del autor sobre su obra. Son muy importantes porque protegen la relación entre el creador y su creación.
 - **Derecho de divulgación:** El autor decide si su obra se publica o no, y de qué forma.
 - **Derecho de paternidad:** El autor tiene derecho a ser reconocido como creador de la obra.
 - **Derecho de la integridad:** Permite impedir deformaciones o modificaciones que dañen la obra o perjudiquen la reputación del autor.
 - **Derecho de modificación:** El autor puede modificar su obra, respetando los derechos adquiridos por terceros.

- Derecho de retirada: Puede retirar la obra del comercio por cambio de convicciones intelectuales o morales.
- Derechos patrimoniales (Económicos): Son los derechos que permiten ganar dinero con la obra. La LPI establece que la propiedad intelectual incluye derechos personales y patrimoniales, y da al autor la plena disposición y el derecho exclusivo de explotación de la obra.
 - Reproducción: El derecho de reproducción permite al autor controlar si su obra puede ser copiada total o parcialmente. Por ejemplo, imprimir un libro o descargar una imagen. Este derecho es importante porque evita que otras personas hagan copias sin permiso y se beneficien del trabajo del autor.
 - Distribución: El derecho de distribución se refiere a poner la obra a disposición del público mediante la venta, alquiler, préstamo o entrega de ejemplares físicos o digitales. Por ejemplo, vender libros, discos o películas.
 - Comunicación pública: La comunicación pública ocurre cuando una obra se presenta o se difunde ante un público sin entregar copias físicas de ella. Algunos ejemplos son emitir una canción en la radio, proyectar una película en un cine o subir un vídeo a internet.
 - Transformación: El derecho de transformación permite al autor decidir si su obra puede ser modificada para crear una obra nueva. Por ejemplo, adaptar una novela al cine o traducir un libro a otro idioma requiere autorización del titular de los derechos. La LPI reconoce que la obra transformada puede tener sus propios derechos, pero siempre respetando los derechos del autor de la obra original.
 - Generalmente hasta 70 años después de la muerte del autor

3.1.4. Copyright vs Patente

El copyright o derecho de autor protege la forma concreta en la que se expresa una creación. Es decir, protege obras como libros, canciones, fotografías, películas, diseños, programas informáticos o textos. En informática, por ejemplo, el copyright puede proteger el código fuente de un programa, su documentación o elementos visuales originales, pero no protege la idea general que hay detrás.

Una patente, en cambio, protege una invención técnica nueva, como un procedimiento, una máquina, un dispositivo o una solución técnica a un problema. Para que algo pueda patentarse, normalmente debe ser nuevo, tener actividad inventiva y poder aplicarse industrialmente. En el ámbito tecnológico, una patente podría proteger un método técnico concreto o un sistema innovador, pero no una simple idea abstracta.

En resumen, el copyright protege creaciones expresadas, mientras que la patente protege invenciones técnicas. Por ejemplo, si una persona desarrolla una aplicación, el copyright puede proteger el código que ha escrito, pero una patente solo protegería una solución técnica novedosa si cumple los requisitos legales.

3.1.5. Infracciones y consecuencias

Una infracción de la LPI se produce cuando una persona utiliza una obra protegida sin permiso del autor o del titular de los derechos. Esto puede ocurrir, por ejemplo, al copiar, distribuir, modificar, vender o publicar una obra sin autorización. En el ámbito digital, también puede pasar al usar imágenes de internet sin licencia, compartir software pirateado, copiar código fuente de otro proyecto o subir contenido protegido a una página web.

Las infracciones pueden tener distintas consecuencias legales y económicas. El autor o titular de los derechos puede exigir que se deje de usar la obra, que se retire el contenido publicado y, en algunos casos, reclamar una compensación económica por los daños causados. Además, si la infracción es grave, intencionada o con ánimo de lucro, puede llegar a tener consecuencias más serias.

En el ámbito informático, esto es especialmente importante porque muchas veces se trabaja con software, librerías, bases de datos, imágenes, textos o contenidos digitales creados por terceros. Por eso, antes de utilizar cualquier recurso, conviene comprobar si tiene licencia, si permite su uso comercial, si exige citar al autor o si tiene restricciones de modificación o distribución.

3.2. RGPD

3.2.1. ¿Qué es el RGPD?

El RGPD es una regulación de la Unión Europea que protege los datos personales y otorga a los individuos un mayor control sobre su información personal. Permite que las personas controlen mejor sus datos personales.

Regula cómo deben recogerse, tratarse, almacenar y protegerse los datos personales de las personas. Entró en aplicación en 2018 y afecta a empresas, administraciones y organizaciones que manejen información identificable, como nombres, correos electrónicos, direcciones IP, historiales médicos o datos de localización.

Es un Reglamento europeo, lo que significa que es de aplicación directa en todos los países de la UE sin necesidad de transposición nacional.

Su finalidad es:

- Proteger la privacidad y derechos de los ciudadanos
- Dar control total al usuario sobre sus datos
- Establecer normas claras para empresas y organizaciones

3.2.2. Principios fundamentales

En el RGPD, los principios fundamentales son las reglas básicas que guían cómo deben tratarse los datos personales. Sirven para asegurar que las empresas y organizaciones no solo usen los datos de forma útil, sino también legal, segura y respetuosa con los derechos de los usuarios.

1. **Licitud, lealtad y transparencia:** Los datos deben tratarse de forma legal y clara para el usuario. La persona debe saber qué información se recoge, para qué se usa y con qué base legal.
2. **Limitación de la finalidad:** Los datos solo pueden utilizarse para el propósito concreto para el que fueron recogidos. No se deberían reutilizar para otros fines sin informar al usuario o sin una base legal válida.
3. **Minimización de datos:** Solo se deben recoger los datos estrictamente necesarios. Desde el diseño de una aplicación, esto implica evitar pedir información innecesaria “por si acaso”.
4. **Exactitud:** Los datos personales deben ser correctos y estar actualizados. Si son erróneos, la organización debe permitir corregirlos o eliminarlos.
5. **Limitación del plazo de conservación:** Los datos no deben guardarse más tiempo del necesario. Cuando ya no hagan falta, deben eliminarse o anonimizarse, salvo que exista una obligación legal para conservarlos.
6. **Integridad y confidencialidad:** La organización debe proteger los datos frente a accesos no autorizados, pérdidas o filtraciones. Esto implica aplicar medidas de seguridad como cifrado, control de accesos y buenas prácticas de desarrollo.
7. **Responsabilidad proactiva:** La empresa no solo debe cumplir el RGPD, sino también poder demostrarlo. Para ello debe documentar sus medidas, revisar sus procesos y aplicar la privacidad desde el diseño del sistema.

3.2.3. Derechos ARSULIPO

Los derechos ARSULIPO son los derechos que tienen los ciudadanos de la UE para controlar sus datos personales. Básicamente, permiten saber qué información tiene una empresa sobre ti, corregirla, borrarla o decidir cómo puede utilizarse.

- **Acceso:** permite pedir a una empresa que te diga qué datos personales tiene sobre ti, para qué los usa, de dónde los ha obtenido y con quién los comparte.
- **Rectificación:** permite corregir datos que sean incorrectos o estén desactualizados, por ejemplo, un número de teléfono, una dirección o un apellido mal escrito.
- **Supresión:** también conocido como derecho al olvido, permite solicitar que se eliminen tus datos cuando ya no sean necesarios, cuando retires tu consentimiento o cuando se hayan usado de forma indebida.
- **Limitación:** permite pedir que tus datos no se usen temporalmente en ciertos casos, por ejemplo, mientras se comprueba si son correctos o si el tratamiento es legal.
- **Portabilidad:** permite recibir tus datos en un formato digital común y reutilizable, o pedir que se envíen a otra empresa. Por ejemplo, cambiar tus datos de un servicio a otro.
- **Oposición:** permite negarte a que tus datos se usen para ciertos fines, como publicidad personalizada, perfiles automáticos o tratamientos basados en intereses legítimos de la empresa.

3.2.4. ¿Cómo cumplir el RGPD?

Para cumplir el RGPD, una empresa o aplicación debe seguir una serie de pasos prácticos que aseguren que los datos personales se tratan de forma legal, segura y transparente.

1. **Identificar qué datos se recogen:** Primero hay que saber qué datos personales se piden al usuario: nombre, correo, teléfono, IP, ubicación, datos bancarios, etc. También hay que tener claro dónde se guardan y quién puede acceder a ellos.
2. **Informar al usuario:** El usuario debe saber qué datos se recogen, para qué se usan, durante cuánto tiempo se conservan y si se comparten con terceros. Esta información debe aparecer en una política de privacidad clara y fácil de entender.
3. **Obtener consentimiento válido:** Cuando sea necesario pedir consentimiento, este debe ser libre, específico, informado y claro. Por ejemplo, no vale marcar una casilla automáticamente; el usuario debe aceptar de forma activa.
4. **Recoger solo los datos necesarios:** Se debe aplicar el principio de minimización. Esto significa no pedir datos “por si acaso”, sino únicamente los que sean necesarios para prestar el servicio.
5. **Proteger los datos con seguridad:** La empresa debe aplicar medidas técnicas y organizativas para evitar robos, pérdidas o accesos no autorizados. Por ejemplo: cifrado, contraseñas seguras, control de permisos, copias de seguridad y actualizaciones.
6. **Permitir ejercer los derechos fácilmente:** El usuario debe poder solicitar acceso a modificar, eliminar, limitar, cambiar de lugar o negar sus datos. Además, la empresa debe responder en un plazo razonable y facilitar el proceso.
7. **Controlar a los proveedores:** Si se usan servicios externos, como hosting, analítica, pasarelas de pago o email marketing, hay que asegurarse de que también cumplen el RGPD. Normalmente esto se regula mediante contratos de tratamiento de datos.
8. **Documentar todo:** La empresa debe guardar pruebas de que cumple el RGPD: consentimientos, políticas, medidas de seguridad, contratos con proveedores y registros de tratamiento. Esto es clave si hay una auditoría o una inspección.

3.2.5. Ventajas de cumplir el RGPD

Implementar correctamente el RGPD tiene como principal ventaja que aumenta la confianza de los usuarios. Cuando una empresa informa de forma clara, protege bien los datos y permite ejercer derechos fácilmente, transmite seriedad y responsabilidad. Esto mejora la imagen de la organización y puede hacer que los clientes se sientan más seguros al usar sus servicios.

Además, cumplir el RGPD ayuda a reducir riesgos legales, económicos y de seguridad. Al tener los datos bien organizados, protegidos y documentados, la empresa está mejor preparada ante auditorías, incidentes o posibles reclamaciones.

3.3. Cumplimiento de Licencias (ISO/IEC 5230)

En el desarrollo de software moderno, la utilización de componentes de terceros y de código abierto es una práctica estándar que acelera la innovación; sin embargo, conlleva

responsabilidades legales que no pueden ignorarse. La **ISO/IEC 5230**, establece los requisitos para un programa de cumplimiento de licencias de software de código abierto de alta calidad.

El objetivo de este apartado es analizar cómo la adopción de este marco permite a la organización identificar, gestionar y mitigar los riesgos asociados a la propiedad intelectual. A través de una comprensión de que son los derechos de autor y la categorización de las licencias, la empresa puede asegurar que sus productos finales cumplan con todas las obligaciones legales, protegiendo tanto su reputación como su patrimonio tecnológico.

3.3.1. Tipos básicos de licencias de software

Licencias de dominio público: Estas licencias no tienen restricciones de derechos de autor. Se pueden usar, modificar y distribuir libremente. En realidad, y técnicamente, no se puede considerar como una licencia, si no la ausencia de ella. Puedes usarlo para lo que quieras, incluso decir que es tuyo o cerrarlo por completo sin dar explicaciones a nadie. Un ejemplo famoso de software en el dominio público es la base de datos SQLite.

Licencias propietarias (Proprietary): Son las licencias que se aplican al software que pertenece a una empresa, con acceso restringido al código. Normalmente, cuando alguien usa software propietario, debe aceptar el acuerdo de licencia de usuario final (End-User License Agreement, EULA).

Licencias de código abierto (Open source): Las licencias de código abierto permiten ver, modificar y distribuir el código. Existen diferentes tipos de licencias de código abierto, con sus propias normas. Linux es un gran ejemplo de licencia Open Source.

Hay licencias de código abierto muy poco restrictivas, como Apache License 2.0. Esta no exige liberar el código de obras derivadas, y puede usarse con fines comerciales, pero sí que tiene restricciones mínimas para la modificación y distribución. Estas son:

- Incluir una copia de la licencia.
- Mantener avisos de copyright, patente y marcas.
- Documentar cambios significativos.

3.3.2. Licencias Permisivas y su impacto comercial

Las licencias permisivas representan el modelo de código abierto más alineado con los intereses de comercialización de software privado. Su característica principal es que ofrecen una libertad casi total de uso, modificación y distribución sin imponer la obligación de liberar el código fuente de las obras derivadas. En el ámbito empresarial, esto permite integrar librerías externas en productos comerciales de código cerrado, protegiendo así la propiedad intelectual y el valor de mercado del software propio.

Un ejemplo es la **Apache License 2.0**, mencionada previamente, la cual, a pesar de ser altamente permisiva y permitir fines comerciales, establece una serie de obligaciones mínimas que la empresa debe gestionar para evitar infracciones legales:

- **Atribución y Transparencia:** Es obligatorio incluir una copia íntegra de la licencia original en cualquier distribución del software.
- **Integridad de la Propiedad:** Se deben mantener todos los avisos originales de copyright, patentes y marcas comerciales presentes en el código fuente.
- **Trazabilidad de Modificaciones:** La licencia exige documentar cualquier cambio significativo realizado sobre el software original, deslindando la responsabilidad del autor inicial sobre las nuevas versiones.

El impacto comercial de este modelo es positivo, ya que reduce los costes de desarrollo al permitir el uso de componentes probados por la comunidad, siempre que se mantenga una política de cumplimiento técnico que asegure que estos avisos y documentos acompañen siempre al producto final.

3.3.3. Licencias Copyleft y la protección del código derivado

El modelo **Copyleft** introduce una filosofía de reciprocidad que impacta directamente en la estrategia de propiedad intelectual de la empresa. A diferencia de las licencias permisivas, licencias como la **GNU General Public License (GPL)** exigen que cualquier trabajo derivado o producto que incorpore dicho código sea distribuido obligatoriamente bajo la misma licencia. Este mecanismo asegura que las mejoras y modificaciones permanezcan en el ecosistema abierto, pero supone un riesgo crítico si se mezcla accidentalmente con código propietario que la empresa desea mantener bajo acceso restringido.

Para mitigar este impacto en entornos donde se requiere proteger la inversión en desarrollo propio, surge la **GNU Lesser General Public License (LGPL)** como una solución intermedia:

La LGPL permite que un proyecto se mantenga como software propietario y cerrado, incluso si utiliza librerías bajo esta licencia. Esta flexibilidad se mantiene siempre que el uso se limite a componentes externos y no se sobrepase un límite de integración profunda que convierta al software principal en una obra derivada.

- **Protección del Código Original:** Cualquier modificación directa sobre la librería LGPL sí debe ser liberada bajo los mismos términos, pero el resto de la aplicación empresarial permanece protegida bajo su propia licencia comercial.

La gestión de estas licencias dentro de una organización exige una auditoría constante para diferenciar qué componentes son puramente copyleft y cuáles permiten un enlace "débil", garantizando que la arquitectura del software no comprometa legalmente la exclusividad del código fuente de la compañía.

3.3.4. Derechos de autor y marcos legales de protección

El software, como obra intelectual, está estrictamente protegido por la **ley de derechos de autor (copyright)**. Este marco legal no es una mera formalidad, sino que otorga a los creadores originales una serie de **derechos de exclusividad** sobre su obra.

Estos derechos permiten al autor controlar quién puede reproducir, distribuir o modificar el código, y son la base sobre la cual se construyen los modelos de licencias. Sin una licencia que otorgue permisos específicos, cualquier uso del software podría constituir una infracción legal.

Dentro de este marco, es fundamental entender que las licencias actúan como contratos que definen los **derechos y deberes** de los usuarios. Sin embargo, la protección de los derechos de autor tiene contrapesos importantes:

- **Límites de la licencia por leyes superiores:** El alcance de una licencia no es ilimitado; las leyes nacionales de derechos de los consumidores pueden prevalecer sobre lo estipulado en el contrato.
- **Invalidez de cláusulas:** Si un desarrollador incluye términos que vulneran los derechos básicos del consumidor, esas anotaciones no se consideran válidas legalmente.
- **Mecanismo de protección al usuario:** Estos marcos legales externos actúan como un recurso valioso para proteger a los usuarios finales y empresas frente a posibles abusos o restricciones excesivas en las licencias propietarias o abiertas.

En resumen, mientras que el copyright protege la propiedad del creador, los marcos legales de protección al consumidor garantizan que el uso del software se mantenga dentro de un equilibrio justo, invalidando condiciones que contradigan la normativa vigente.

3.3.5. Gestión de riesgos y toma de decisiones

La gestión eficiente del software requiere que los consumidores y empresas sean plenamente conscientes de sus derechos y deberes antes de la implementación. Para minimizar riesgos legales y técnicos, es fundamental:

- **Revisión exhaustiva:** Analizar siempre los documentos de licencia antes de usar cualquier software para tomar decisiones basadas en información real.
- **Diferenciación de modelos:** Identificar si se trata de software de dominio público (sin restricciones), propietario (con EULA restringido) o de código abierto (con normas específicas de modificación).
- **Cumplimiento de obligaciones:** En licencias abiertas, asegurar el mantenimiento de avisos de copyright y la documentación de cambios para evitar infracciones que comprometan la propiedad intelectual de la organización.

3.4. Ética y Responsabilidad Civil: ISO 42001

La rápida integración de la Inteligencia Artificial (IA) en los procesos de negocio ha generado la necesidad de un marco de gobernanza que trascienda la eficiencia técnica para centrarse en la seguridad y la integridad. La **ISO/IEC 42001** representa el primer

estándar internacional para establecer **un Sistema de Gestión de Inteligencia Artificial (SGIA)**, proporcionando una estructura robusta para gestionar los riesgos y oportunidades de esta tecnología.

3.4.1. Fundamentos del SGIA: Gestión y Gobernanza

El **Sistema de Gestión de Inteligencia Artificial (SGIA)** propuesto por la ISO 42001 no es una herramienta de software, sino un marco organizativo o manual de buenas prácticas que establece un orden en el uso para el desarrollo de la IA, evitando que el despliegue tecnológico se realice sin un control.

La implementación de esta norma aporta orden, seguridad en la protección de datos y una imagen de profesionalidad ante el mercado.

3.4.2. Desafíos Técnicos de la IA

La naturaleza intrínseca de la IA introduce retos que el software tradicional no posee, principalmente debido a su capacidad de operar fuera de la lógica humana. El fenómeno de la "**Caja Negra**" representa uno de los mayores desafíos, ya que muchos modelos de *Deep Learning* llegan a conclusiones sin que sea posible trazar fácilmente el camino lógico seguido. Esto genera una necesidad crítica de transparencia para evitar que la empresa dependa de decisiones "mágicas" o inexplicables.

Además, la IA va evolucionando; a diferencia de un código estático, el modelo sigue aprendiendo y modificando su comportamiento a medida que ingiere nuevos datos, lo que puede derivar en una degradación del rendimiento o en un cambio de criterio no deseado si no existe una supervisión constante.

Finalmente, el problema de los **sesgos** ("el espejo sucio") implica que la IA es tan buena como los datos que la alimentan: si los datos de entrenamiento contienen prejuicios históricos o estadísticos, el algoritmo los replicará y amplificará, convirtiendo un error técnico en una discriminación sistémica.

3.4.3. Impacto y aplicabilidad

La gestión proactiva del riesgo constituye el núcleo operativo de la norma **ISO/IEC 42001**. Antes de proceder a la implementación o despliegue de cualquier sistema de Inteligencia Artificial, es crucial realizar una **evaluación del impacto**. Es un proceso donde el equipo de desarrollo y los responsables de cumplimiento se plantean la pregunta: "¿A quién podemos perjudicar con esta tecnología?". Esta evaluación permite medir la **incertidumbre** y determinar si los beneficios del sistema compensan los riesgos potenciales para los derechos de las personas, la seguridad de los datos o la estabilidad de la propia organización.

Como resultado de este análisis de riesgos, se genera la **Declaración de Aplicabilidad (SoA)**. Este documento técnico es crucial para la gobernanza, ya que actúa como una "lista de la compra" personalizada para la empresa basada en:

- **Selección de Controles:** La organización detalla específicamente qué reglas y controles de la norma ISO va a adoptar basándose en los riesgos identificados previamente.
- **Justificación de Exclusiones:** De igual manera, se debe argumentar técnicamente por qué ciertos controles no son aplicables al proyecto en cuestión.
- **Responsabilidad técnica:** Al formalizar qué se aplica y qué no, la empresa deja constancia de que no está utilizando la norma de forma genérica, sino que ha diseñado un marco de seguridad a medida de su realidad operativa.

En definitiva, actuar con responsabilidad bajo el marco de la **ISO 42001** implica sentarse a evaluar las consecuencias antes de lanzar la IA, asegurando que cada decisión técnica esté respaldada por una evaluación de riesgos documentada y una declaración de aplicabilidad coherente con los objetivos éticos y legales de la empresa.

3.4.4. Responsabilidad Civil del Ingeniero y Trazabilidad Legal (ByDesign)

La responsabilidad civil se refiere a la obligación de reparar daños causados por el software.

- **Responsabilidad "by design":** El cumplimiento legal comienza desde la primera línea de código o la selección del modelo; no es un paso final.
- **Trazabilidad:** La ISO 42001 obliga a mantener registros detallados (logs) de todo el proceso. En caso de conflicto legal, esta trazabilidad sirve como prueba de que se siguieron las buenas prácticas y se actuó con diligencia profesional.
- **Vínculo con LPI y RGPD:** El ingeniero debe asegurar que el uso de datos respeta la propiedad intelectual de terceros y la protección de datos personales para evitar sanciones civiles y administrativas.

4. La documentación como línea de defensa

A lo largo de este documento se han tratado dos bloques que a primera vista pueden parecer independientes: cómo se documenta bien el software y cómo se cumple con el marco legal y ético que lo rodea. Sin embargo, la conclusión más importante es que ambos bloques son inseparables. La documentación no es solo una buena práctica de ingeniería; es también el principal mecanismo de protección legal y ética del que dispone un equipo de desarrollo.

4.1. Docs as Code e ISO 26514 como base del cumplimiento

Adoptar Docs as Code no es solo una decisión de productividad. Es una decisión de gobernanza. Un pipeline de CI/CD que genera, valida y publica documentación automáticamente en cada merge garantiza que la documentación refleja el estado real del sistema en todo momento. Eso es exactamente lo que exigen la ISO 5230 cuando pide un BOM actualizado, el RGPD cuando exige registros de tratamiento

documentados, o la ISO 42001 cuando requiere trazabilidad sobre las decisiones del sistema.

La ISO 26514, por su parte, asegura que esa documentación no solo existe, sino que es útil: orientada al usuario, validada y estructurada. Un documento que nadie entiende no sirve como evidencia ni como herramienta.

4.2. Las licencias, la LPI y el RGPD no se gestionan de memoria

Los tres marcos legales tratados en este documento tienen algo en común: exigen documentación como requisito de cumplimiento, no como recomendación. La ISO 5230 requiere un Bill of Materials con todas las licencias identificadas. La LPI protege el código desde el momento en que se escribe, pero esa protección solo puede ejercerse si hay registros que acrediten la autoría y la fecha. El RGPD exige un registro de actividades de tratamiento que debe estar disponible ante cualquier inspección.

En los tres casos, la ausencia de documentación no es una cuestión menor: es la diferencia entre poder demostrar el cumplimiento o no poder hacerlo.

4.3. El factor humano: el bus factor y la responsabilidad civil

La ISO 25010 introduce el concepto de mantenibilidad como característica de calidad del software, y dentro de ella, el riesgo del bus factor: el conocimiento crítico que solo existe en la cabeza de una persona. Cuando esa persona no está disponible, el sistema se vuelve opaco e inmantenible.

Ese mismo riesgo tiene una lectura legal. Si un ingeniero es el único que conoce cómo funciona un módulo que toma decisiones automatizadas, y ese módulo causa un daño, la cadena de responsabilidad civil apunta directamente a esa persona. La documentación distribuye ese conocimiento, y con él, distribuye también la responsabilidad de forma justa y trazable.

5. Conclusión

Documentar bien no es burocracia. Es la forma más concreta y práctica de proteger al equipo, al producto y a los usuarios. A lo largo de este documento se ha visto que los problemas de documentación y los problemas legales raramente son independientes: una licencia mal gestionada, un tratamiento de datos sin registrar o una decisión automatizada sin trazabilidad son, antes que nada, fallos de documentación. No porque alguien haya incumplido una ley conscientemente, sino porque nadie dejó constancia de que se había tomado la decisión correcta.

Los estándares vistos en este documento, desde la filosofía Docs as Code hasta la ISO 42001, pasando por la ISO 26514, la ISO 5230, la LPI y el RGPD, convergen en una misma idea: el software de calidad es aquel que puede explicarse, auditarse y mantenerse. No solo por quienes lo construyeron, sino por cualquier persona que tenga que trabajar con él en el futuro, ya sea un desarrollador que incorpora una nueva funcionalidad, un auditor que verifica el cumplimiento normativo, o un juez que evalúa una responsabilidad civil.

En ese sentido, la documentación no es el resultado final del proceso de ingeniería, sino una parte activa de él. Un equipo que documenta mientras desarrolla, que versiona su documentación junto al código y que la mantiene actualizada en cada entrega, no solo trabaja mejor: trabaja de forma más segura. Reduce su exposición legal, facilita la incorporación de nuevos miembros, acelera las auditorías y genera confianza en los usuarios y clientes del producto.

La conclusión, en definitiva, es sencilla: el mejor momento para documentar era al principio del proyecto. El segundo mejor momento es ahora.

6. Bibliografía

Agencia Española de Protección de Datos. (s. f.). *Ejerce tus derechos*. AEPD.
https://www.aepd.es/derechos-y-deberes/ejerce-tus-derechos?utm_source

Unión Europea. (2016). *Reglamento (UE) 2016/679, Reglamento General de Protección de Datos*. https://eur-lex.europa.eu/eli/reg/2016/679/oj?utm_source

Real Decreto Legislativo 1/1996 (Ley de Propiedad Intelectual).
<https://boe.es/buscar/act.php?id=BOE-A-1996-8930>

ISO/IEC/IEEE 26514:2022: [ISO/IEC/IEEE 26514:2022 - Systems and software engineering — Design and development of information for users](#)

ISO 26514, Carsten Sieber: [The New ISO/IEC/IEEE 26514:2022-01 – Whatever Works is Allowed](#)

AENOR ISO 42001-Gestión Inteligencia Artificial: [ISO 42001 Gestión Inteligencia Artificial - AENOR](#)

ISO 5230: [OpenChain ISO/IEC 5230 – License Compliance – OpenChain](#)

Vídeo Clave- Licencias: [What Are The Different Types Of Software Licenses? - Consumer Laws For You](#)

7. Anexo

7.1. Documento de Arquitectura ISO 26514

Plantilla para Documento de Arquitectura

Información del documento

Nombre del proyecto	
Versión	
Fecha de creación	
Última actualización	
Autor(es)	
Revisado por	

1. Introducción

1.1. Propósito

Describir brevemente para qué sirve este documento y qué decisiones arquitectónicas recoge.

1.2. Alcance

Indicar qué partes del sistema cubre este documento y cuáles quedan fuera de su ámbito.

1.3. Audiencia objetivo

Identificar a quién va dirigido este documento: desarrolladores, arquitectos, responsables de producto, auditores, etc. Ser específico: no "personal técnico", sino roles concretos.

1.4. Documentos relacionados

Lista de documentos de referencia: especificaciones de requisitos, diagramas de base de datos, contratos de API, etc.

2. Descripción general del sistema

2.1. Contexto y problema que resuelve

Explicar en lenguaje claro qué problema resuelve el sistema y por qué existe. No asumir que el lector conoce el contexto.

2.2. Restricciones y condicionantes

Restricciones técnicas, legales, de negocio o de equipo que han condicionado las decisiones arquitectónicas. Por ejemplo: lenguaje impuesto, plazo fijo, normativa aplicable, infraestructura existente.

2.3. Suposiciones

Decisiones tomadas sobre la base de información no confirmada. Si alguna suposición cambia, indicar qué partes del documento quedan afectadas.

3. Arquitectura del sistema

3.1. Visión general

Diagrama de alto nivel del sistema. Puede ser un diagrama de componentes, de contexto C4 o cualquier representación que muestre las piezas principales y sus relaciones.

3.2. Componentes principales

Componente	Responsabilidad	Tecnología	Dependencias

3.3. Diagrama de despliegue

Representación de cómo se despliega el sistema: servidores, contenedores, servicios en la nube, redes, etc.

3.4. Flujos principales

Descripción de los flujos de datos o de control más importantes del sistema. Un diagrama de secuencia o de flujo por cada proceso crítico.

4. Decisiones arquitectónicas

Registro de las decisiones de diseño relevantes. Para cada una, documentar el contexto, las alternativas consideradas y la razón por la que se eligió la opción adoptada.

5. Atributos de calidad (ISO 25010)

Atributo	Estrategia adoptada	Métrica
Mantenibilidad		
Seguridad		
Rendimiento		
Disponibilidad		
Portabilidad		

6. Interfaces y APIs

6.1. APIs expuestas

Listado de endpoints o contratos que el sistema expone a otros sistemas o clientes.

6.2. APIs consumidas

Dependencias externas: servicios de terceros, APIs internas de otros equipos, etc.

6.3. Formato de intercambio

Protocolo, formato de datos y versión: REST/JSON, gRPC, GraphQL, etc.

7. Seguridad y cumplimiento normativo

7.1. Modelo de autenticación y autorización

Mecanismo de control de acceso: JWT, OAuth 2.0, RBAC, etc.

7.2. Datos personales tratados

Listado de datos personales que maneja el sistema y base legal de su tratamiento, conforme al RGPD.

7.3. Componentes open source utilizados

Bill of Materials simplificado: librería, versión y licencia. Referencia al BOM completo si existe.

8. Registro de cambios

Versión	Fecha	Autor	Descripción
1.0			

7.2. Contrato de Cesión de Derecho

Plantilla para Contrato de Cesión de Derechos de Software

En [ciudad], a [día] de [mes] de [año].

REUNIDOS

De una parte, **D./Dña. [nombre completo]**, con DNI/NIF número [número], con domicilio a efectos de notificaciones en [dirección completa], en adelante denominado el **CEDENTE**.

De otra parte, **[nombre de la empresa o persona física]**, con CIF/NIF número [número], con domicilio social en [dirección completa], representada en este acto por D./Dña. [nombre del representante] en calidad de [cargo], en adelante denominado el **CESIONARIO**.

Ambas partes se reconocen mutuamente capacidad legal suficiente para contratar y obligarse, y a tal efecto:

EXPONEN

I. Que el CEDENTE es autor y titular originario de los derechos de propiedad intelectual sobre el software denominado **[nombre del software]**, en adelante "**la Obra**", descrito en el Anexo I del presente contrato.

II. Que la Obra ha sido creada de forma original por el CEDENTE, quien declara que no está gravada con cargas, limitaciones ni derechos de terceros que pudieran impedir o menoscabar la cesión objeto del presente contrato.

III. Que el CESIONARIO tiene interés en adquirir los derechos de explotación sobre la Obra en los términos que a continuación se establecen.

Reconociéndose ambas partes la capacidad necesaria para suscribir el presente contrato, acuerdan las siguientes:

CLÁUSULAS

Primera. Objeto del contrato

El CEDENTE cede al CESIONARIO, en los términos establecidos en el presente contrato, los derechos de explotación sobre la Obra descrita en el Anexo I, de conformidad con lo dispuesto en el Título III del Real Decreto Legislativo 1/1996, de 12 de abril, por el que se aprueba el Texto Refundido de la Ley de Propiedad Intelectual.

Segunda. Derechos objeto de cesión

La cesión comprende, con el alcance y las limitaciones que se indican en cada caso, los siguientes derechos:

- **Reproducción:** derecho a copiar y reproducir la Obra en cualquier soporte o formato, incluyendo su almacenamiento en servidores propios o de terceros.
- **Distribución:** derecho a poner a disposición del público la Obra o copias de la misma, en cualquier modalidad.
- **Comunicación pública:** derecho a poner la Obra a disposición de terceros a través de redes digitales, incluyendo internet.
- **Transformación:** derecho a modificar, adaptar, traducir o crear obras derivadas a partir de la Obra original. *(Indicar si se incluye o excluye este derecho.)*

Tercera. Exclusividad

(Seleccionar la opción aplicable y eliminar la otra.)

- La cesión se realiza con carácter **exclusivo**, por lo que el CEDENTE no podrá ceder los mismos derechos a terceros durante la vigencia del presente contrato.
- La cesión se realiza con carácter **no exclusivo**, pudiendo el CEDENTE ceder los mismos derechos a terceros.

Cuarta. Ámbito territorial

La presente cesión tiene carácter **[mundial / limitado al territorio de: especificar]**.

Quinta. Duración

La presente cesión tendrá una duración de **[duración]**, con inicio en la fecha de firma del presente contrato. *(Alternativa: "con carácter indefinido, hasta el límite legal establecido por la LPI.")*

Transcurrido el plazo, los derechos cedidos revertirán automáticamente al CEDENTE, salvo acuerdo expreso de renovación entre las partes.

Sexta. Contraprestación económica

(Seleccionar y completar la opción aplicable.)

- En contraprestación por la cesión de derechos, el CESIONARIO abonará al CEDENTE la cantidad de **[importe] euros**, mediante [forma de pago], en el plazo de [plazo] desde la firma del presente contrato.
- La presente cesión se realiza a título **gratuito**, sin contraprestación económica para el CEDENTE.

Séptima. Entrega de la Obra

El CEDENTE se compromete a entregar al CESIONARIO, en el plazo de [plazo] desde la firma del presente contrato, los siguientes materiales:

- Código fuente completo de la Obra, en formato [especificar].
- Documentación técnica asociada, según se detalla en el Anexo I.
- Credenciales de acceso a repositorios, entornos o servicios vinculados a la Obra, si los hubiera.

Octava. Garantías del cedente

El CEDENTE garantiza que:

- Es el único autor y titular de los derechos sobre la Obra, o cuenta con las autorizaciones necesarias de los coautores.
- La Obra no vulnera derechos de propiedad intelectual de terceros.
- La Obra no incorpora componentes con licencias incompatibles con los términos de la presente cesión.

- En caso de que la Obra incluya componentes open source, estos se detallan en el Anexo II junto con sus licencias correspondientes.

Novena. Derechos morales

De conformidad con el artículo 14 de la LPI, el CEDENTE conserva en todo caso sus derechos morales sobre la Obra, en particular el derecho a ser reconocido como autor. El CESIONARIO se compromete a mantener los avisos de autoría en la Obra y en sus versiones derivadas.

Décima. Confidencialidad

Ambas partes se comprometen a mantener la confidencialidad sobre el contenido del presente contrato y sobre cualquier información técnica o comercial intercambiada en el marco del mismo, durante la vigencia del contrato y por un plazo de [plazo] tras su extinción.

Undécima. Resolución del contrato

El presente contrato podrá resolverse por cualquiera de las siguientes causas:

- Incumplimiento grave de cualquiera de las obligaciones establecidas en el mismo.
- Mutuo acuerdo entre las partes, formalizado por escrito.
- Las causas legalmente establecidas en la LPI y en el Código Civil.

Duodécima. Legislación aplicable y jurisdicción

El presente contrato se rige por la legislación española, en particular por el Real Decreto Legislativo 1/1996, de 12 de abril. Para la resolución de cualquier controversia derivada del mismo, las partes se someten, con renuncia a su propio fuero si lo tuvieran, a los Juzgados y Tribunales de [ciudad].

Y en prueba de conformidad, ambas partes firman el presente contrato por duplicado en el lugar y fecha indicados en el encabezamiento.

FIRMA CEDENTE

FIRMA CESIONARIO

